

Budiyanto_HW2

Cynthia Budiyanto

```
# Initial packages required (we'll be adding more)
library(tidyverse)
library(nycflights13)
```

Do not Repeat Yourself: Also known as DRY, if you copy or paste code more than twice, you should write a function instead.

When writing a function, it is usually best to start with the code you know works for one instance, and then “function-ize” it.

Vector functions

Example 1: Rescale variables from 0 to 1.

This code creates a 10 x 4 tibble filled with random values taken from a normal distribution with mean 0 and SD 1

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
df
```

```
# A tibble: 10 x 4
      a      b      c      d
  <dbl> <dbl> <dbl> <dbl>
1  0.798  0.495 -0.465 -1.41
2  0.0870 -0.478  0.602 -1.92
3  0.277 -0.0307 -1.29  0.0846
```

```

4  0.0872  2.16    0.176 -0.0290
5 -0.0968 -2.05   -0.173  1.32
6 -0.0271 -0.730  -0.584 -0.536
7  0.0419 -1.27   -0.141 -1.48
8 -0.137   0.0957  0.182 -0.562
9  0.814   0.441  -0.279  0.925
10 0.182   2.88    0.240 -1.24

```

This code below for rescaling variables from 0 to 1 is ripe for functions... we did it four times!

It's easiest to start with working code and turn it into a function.

```

df$a <- (df$a - min(df$a)) / (max(df$a) - min(df$a))
df$b <- (df$b - min(df$b)) / (max(df$b) - min(df$b))
df$c <- (df$c - min(df$c)) / (max(df$c) - min(df$c))
df$d <- (df$d - min(df$d)) / (max(df$d) - min(df$d))
df

```

```

# A tibble: 10 x 4
      a      b      c      d
  <dbl> <dbl> <dbl> <dbl>
1 0.983  0.515 0.436 0.160
2 0.235  0.318 1      0
3 0.435  0.409 0      0.619
4 0.235  0.854 0.775 0.584
5 0.0419 0      0.590 1
6 0.115  0.267 0.373 0.428
7 0.188  0.158 0.607 0.138
8 0      0.435 0.778 0.420
9 1      0.504 0.534 0.878
10 0.335  1      0.809 0.211

```

Notice first what changes and what stays the same in each line. Then, if we look at the first line above, we see we have one value we're using over and over: **df\$a**. So our function will have one input. We'll start with our code from that line, then replace the input (df\$a) with x. We should give our function a name that explains what it does. The name should be a verb.

```

# I'm going to show you how to write the function in class!
# I have it in the code already below, but don't look yet!
# Let's try to write it together first!

```

```

. . . . .

```

```
# Our function (first draft!)
rescale01 <- function(x) {
  (x - min(x)) / (max(x) - min(x))
}
```

Note the **general form of a function**:

```
name <- function(arguments) {
  body
}
```

Every function contains 3 essential components:

1. A **name**. The name should clearly evoke what the function does; hence, it is often a verb (action). Here we'll use `rescale01` because this function rescales a vector to lie between 0 and 1. `snake_case` is good; `CamelCase` is just okay.
2. The **arguments**. The arguments are things that vary across calls and they are usually nouns - first the data, then other details. Our analysis above tells us that we have just one; we'll call it `x` because this is the conventional name for a numeric vector, but you can use any word.
3. The **body**. The body is the code that's repeated across all the calls. By default a function will return the last statement; use `return()` to specify a return value

Summary: Functions should be written for both humans and computers!

Once we have written a function we like, then we need to test it with different inputs!

```
temp <- c(4, 6, 8, 9)
rescale01(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
temp0 <- c(4, 6, 8, 9, NA)
rescale01(temp0)
```

```
[1] NA NA NA NA NA
```

OK, so `NA`'s don't work the way we want them to.

```
rescale01 <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}
rescale01(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
rescale01(temp0)
```

```
[1] 0.0 0.4 0.8 1.0 NA
```

We can continue to improve our function. Here is another method, which uses the existing `range` function within R to avoid 3 max/min executions:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
rescale01(c(0, 5, 10))
```

```
[1] 0.0 0.5 1.0
```

```
rescale01(c(-10, 0, 10))
```

```
[1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
[1] 0.00 0.25 0.50 NA 1.00
```

We should continue testing unusual inputs. Think carefully about how you want this function to behave... the current behavior is to include the Inf (infinity) value when calculating the range. You get strange output everywhere, but it's pretty clear that there is a problem right away when you use the function. In the example below (rescale1), you ignore the infinity value when calculating the range. The function returns Inf for one value, and sensible stuff for the rest. In many cases this may be useful, but it could also hide a problem until you get deeper into an analysis.

```
x <- c(1:10, Inf)
rescale01(x)
```

```
[1] 0 0 0 0 0 0 0 0 0 0 NaN
```

```
rescale1 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale1(x)
```

```
[1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667
[8] 0.7777778 0.8888889 1.0000000      Inf
```

Now we've used functions to simplify original example. We will learn to simplify further in iterations (Ch 26)

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
# add a little noise
df$a[5] = NA
df$b[6] = Inf
df
```

```
# A tibble: 10 x 4
      a      b      c      d
  <dbl> <dbl> <dbl> <dbl>
1  0.252 -0.727 -1.70  -0.133
```

```

2  0.690 -0.0142 -0.270 -1.07
3 -1.24  -0.552 -0.0172  0.321
4 -0.886  0.273  0.375  0.724
5 NA      1.97   0.375 -2.69
6 -1.59  Inf     1.45   0.631
7  2.28  -1.68  -0.0907 -0.590
8  0.305 -0.0370  0.935 -2.21
9  0.983 -2.11  -0.640  0.740
10 -1.35  -1.48   0.997  0.0281

```

```

df$a_new <- rescale1(df$a)
df$b_new <- rescale1(df$b)
df$c_new <- rescale1(df$c)
df$d_new <- rescale1(df$d)
df

```

```
# A tibble: 10 x 8
```

	a	b	c	d	a_new	b_new	c_new	d_new
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.252	-0.727	-1.70	-0.133	0.476	0.339	0	0.746
2	0.690	-0.0142	-0.270	-1.07	0.589	0.514	0.454	0.473
3	-1.24	-0.552	-0.0172	0.321	0.0898	0.382	0.534	0.878
4	-0.886	0.273	0.375	0.724	0.182	0.584	0.658	0.995
5	NA	1.97	0.375	-2.69	NA	1	0.658	0
6	-1.59	Inf	1.45	0.631	0	Inf	1	0.968
7	2.28	-1.68	-0.0907	-0.590	1	0.105	0.510	0.612
8	0.305	-0.0370	0.935	-2.21	0.490	0.508	0.836	0.140
9	0.983	-2.11	-0.640	0.740	0.665	0	0.336	1
10	-1.35	-1.48	0.997	0.0281	0.0630	0.154	0.856	0.792

```

df |>
  select(1:4) |>
  mutate(a_new = rescale1(a),
         b_new = rescale1(b),
         c_new = rescale1(c),
         d_new = rescale1(d))

```

```
# A tibble: 10 x 8
```

	a	b	c	d	a_new	b_new	c_new	d_new
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	0.252	-0.727	-1.70	-0.133	0.476	0.339	0	0.746

2	0.690	-0.0142	-0.270	-1.07	0.589	0.514	0.454	0.473
3	-1.24	-0.552	-0.0172	0.321	0.0898	0.382	0.534	0.878
4	-0.886	0.273	0.375	0.724	0.182	0.584	0.658	0.995
5	NA	1.97	0.375	-2.69	NA	1	0.658	0
6	-1.59	Inf	1.45	0.631	0	Inf	1	0.968
7	2.28	-1.68	-0.0907	-0.590	1	0.105	0.510	0.612
8	0.305	-0.0370	0.935	-2.21	0.490	0.508	0.836	0.140
9	0.983	-2.11	-0.640	0.740	0.665	0	0.336	1
10	-1.35	-1.48	0.997	0.0281	0.0630	0.154	0.856	0.792

```
# Even better - from Chapter 26
df |>
  select(1:4) |>
  mutate(across(a:d, rescale1, .names = "{.col}_new"))
```

```
# A tibble: 10 x 8
      a         b         c         d   a_new   b_new c_new d_new
  <dbl>   <dbl>   <dbl>   <dbl> <dbl>   <dbl> <dbl> <dbl>
1  0.252 -0.727 -1.70  -0.133  0.476   0.339 0     0.746
2  0.690 -0.0142 -0.270 -1.07   0.589   0.514 0.454 0.473
3 -1.24  -0.552 -0.0172  0.321  0.0898  0.382 0.534 0.878
4 -0.886  0.273  0.375  0.724  0.182   0.584 0.658 0.995
5 NA      1.97  0.375 -2.69   NA      1     0.658 0
6 -1.59  Inf    1.45  0.631  0      Inf    1     0.968
7  2.28  -1.68  -0.0907 -0.590  1      0.105 0.510 0.612
8  0.305 -0.0370  0.935 -2.21   0.490   0.508 0.836 0.140
9  0.983 -2.11  -0.640  0.740  0.665   0     0.336 1
10 -1.35  -1.48  0.997  0.0281 0.0630  0.154 0.856 0.792
```

Options for handling NAs in functions

Before we try some practice problems, let's consider various options for handling NAs in functions. We used the `na.rm` option within functions like `min`, `max`, and `range` in order to take care of missing values. But there are alternative approaches:

- filter/remove the NA values before rescaling
- create an if statement to check if there are NAs; return an error if NAs exist
- create a `removeNAs` option in the function we are creating

Let's take a look at each alternative approach in turn:

Filter/remove the NA values before rescaling

```
df <- tibble(  
  a = rnorm(10),  
  b = rnorm(10),  
  c = rnorm(10),  
  d = rnorm(10)  
)  
df$a[5] = NA  
df
```

```
# A tibble: 10 x 4  
      a      b      c      d  
  <dbl> <dbl> <dbl> <dbl>  
1 -0.292  2.36 -0.662  0.303  
2  0.216 -0.237 -0.797 -0.637  
3  2.53   0.708  0.832  1.72  
4  0.0293 1.02   0.635 -0.892  
5 NA      1.08 -0.975  0.118  
6  0.296  0.533  0.726 -1.27  
7  0.140  1.06   0.245 -0.0623  
8 -0.257 -0.844 -0.562  0.0863  
9 -1.58   -0.333 -1.32   0.284  
10 -0.847  0.949 -0.716  1.31
```

```
rescale_basic <- function(x) {  
  (x - min(x)) / (max(x) - min(x))  
}
```

```
df |>  
  filter(!is.na(a)) |>  
  mutate(new_a = rescale_basic(a))
```

```
# A tibble: 9 x 5  
      a      b      c      d new_a  
  <dbl> <dbl> <dbl> <dbl> <dbl>  
1 -0.292  2.36 -0.662  0.303  0.313  
2  0.216 -0.237 -0.797 -0.637  0.437  
3  2.53   0.708  0.832  1.72   1  
4  0.0293 1.02   0.635 -0.892  0.391  
5  0.296  0.533  0.726 -1.27   0.456
```

```

6  0.140  1.06  0.245 -0.0623 0.418
7 -0.257 -0.844 -0.562  0.0863 0.322
8 -1.58  -0.333 -1.32  0.284  0
9 -0.847  0.949 -0.716  1.31  0.178

```

[Pause to Ponder:] Do you notice anything in the output above that gives you pause?

Since we are filtering before doing the calculation, we remove the NA row first, which then also alter the rescaling range. The scaling is then based on a different dataset than the original column.

Create an if statement to check if there are NAs; return an error if NAs exist

First, here's an example involving weighted means:

```

# Create function to calculate weighted mean
wt_mean <- function(x, w) {
  sum(x * w) / sum(w)
}
wt_mean(c(1, 10), c(1/3, 2/3))

```

```
[1] 7
```

```
wt_mean(1:6, 1:3)
```

```
[1] 7.666667
```

[Pause to Ponder:] Why is the answer to the last call above 7.67? Aren't we taking a weighted mean of 1-6, all of which are below 7?

Since the two vectors do not have equal length (1:6 and 1:3), the shorter one gets recycled.

```

# update function to handle cases where data and weights of unequal length

wt_mean <- function(x, w) {
  if (length(x) != length(w)) {
    stop("`x` and `w` must be the same length", call. = FALSE)
  } else {
    sum(w * x) / sum(w)
  }
}
wt_mean(1:6, 1:3)

```

Error:
! `x` and `w` must be the same length

```
# should produce an error now if weights and data different lengths  
# - nice example of if and else
```

[Pause to Ponder:] What does the `call.` option do?

`x` and `w` must be the same length WHEN FALSE `wt_mean(1:6, 1:3)` : `x` and `w` must be the same length WHEN TRUE

Now let's apply this to our rescaling function

```
rescale_w_error <- function(x) {  
  if (is.na(sum(x))) {  
    stop("`x` cannot have NAs", call. = FALSE)  
  } else {  
    (x - min(x)) / (max(x) - min(x))  
  }  
}  
  
temp <- c(4, 6, 8, 9)  
rescale_w_error(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
temp <- c(4, 6, 8, 9, NA)  
rescale_w_error(temp)
```

Error:
! `x` cannot have NAs

[Pause to Ponder:] Why can't we just use `if (is.na(x))` instead of `is.na(sum(x))`?

`is.na(x)` returns us back a vector, not a single value which what the if function wants

Create a removeNAs option in the function we are creating

```
rescale_NAoption <- function(x, removeNAs = FALSE) {
  (x - min(x, na.rm = removeNAs)) /
  (max(x, na.rm = removeNAs) - min(x, na.rm = removeNAs))
}

temp <- c(4, 6, 8, 9)
rescale_NAoption(temp)
```

```
[1] 0.0 0.4 0.8 1.0
```

```
temp <- c(4, 6, 8, 9, NA)
rescale_NAoption(temp, removeNAs = TRUE)
```

```
[1] 0.0 0.4 0.8 1.0 NA
```

OK, but all the other summary stats functions use `na.rm` as the input, so to be consistent, it's probably better to do something slightly awkward like this:

```
rescale_NAoption <- function(x, na.rm = FALSE) {
  (x - min(x, na.rm = na.rm)) /
  (max(x, na.rm = na.rm) - min(x, na.rm = na.rm))
}

temp <- c(4, 6, 8, 9, NA)
rescale_NAoption(temp, na.rm = TRUE)
```

```
[1] 0.0 0.4 0.8 1.0 NA
```

`wt_mean()` is an example of a “summary function (single value output) instead of a”mutate function” (vector output) like `rescale01()`. Here's another summary function to produce the mean absolute percentage error:

```
mape <- function(actual, predicted) {
  sum(abs((actual - predicted) / actual)) / length(actual)
}

y <- c(2,6,3,8,5)
yhat <- c(2.5, 5.1, 4.4, 7.8, 6.1)
mape(actual = y, predicted = yhat)
```

```
[1] 0.2223333
```

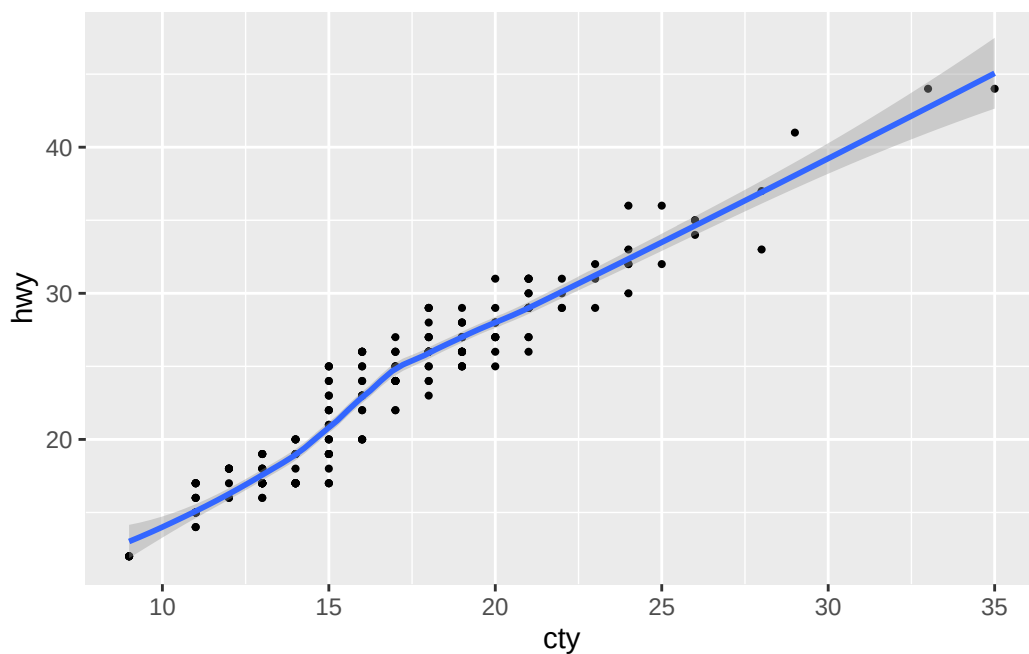
Data frame functions

These work like dplyr verbs, taking a data frame as the first argument, and then returning a data frame or a vector.

Demonstration of tidy evaluation in functions

```
# Start with working code then functionize
ggplot(data = mpg, mapping = aes(x = cty, y = hwy)) +
  geom_point(size = 0.75) +
  geom_smooth()
```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



```
make_plot <- function(dataset, xvar, yvar, pt_size = 0.75) {
  ggplot(data = dataset, mapping = aes(x = xvar, y = yvar)) +
    geom_point(size = pt_size) +
    geom_smooth()
}
```

```
make_plot(dataset = mpg, xvar = cty, yvar = hwy) # Error!
```

```
Error in `geom_point()` :
! Problem while computing aesthetics.
i Error occurred in the 1st layer.
Caused by error:
! object 'cty' not found
```

The problem is tidy evaluation, which makes most common coding easier, but makes some less common things harder. Key terms to understand tidy evaluation:

- env-variables = live in the environment (`mpg`)
- data-variables = live in data frame or tibble (`cty`)
- data masking = tidyverse use data-variables as if they are env-variables. That is, you don't always need `mpg$cty` to access `cty` in tidyverse

The key idea behind data masking is that it blurs the line between the two different meanings of the word “variable”:

- env-variables are “programming” variables that live in an environment. They are usually created with `<-`.
- data-variables are “statistical” variables that live in a data frame. They usually come from data files (e.g. `.csv`, `.xls`), or are created manipulating existing variables.

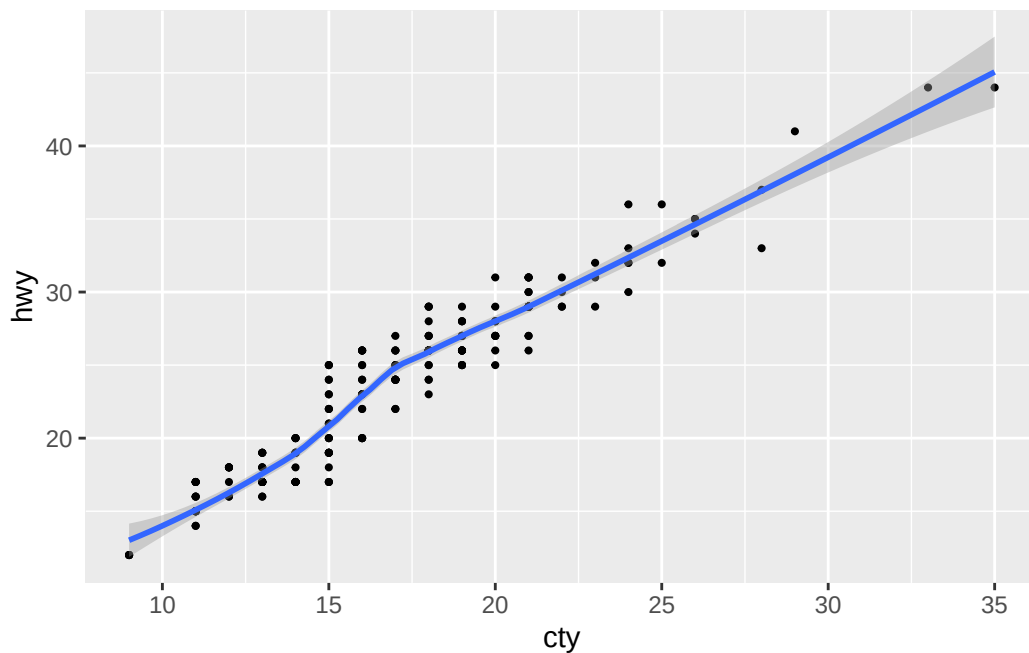
The solution is to embrace `{{ }}` data-variables which are user inputs into functions. One way to remember what's happening, as suggested by our book authors, is to think of `{{ }}` as looking down a tunnel — `{{ var }}` will make a dplyr function look inside of `var` rather than looking for a variable called `var`. Thus, embracing a variable tells dplyr to use the value stored inside the argument, not the argument as the literal variable name.

See Section 25.3 of R4DS for more details (and there are plenty!).

```
# This will work to make our plot!
make_plot <- function(dataset, xvar, yvar, pt_size = 0.75) {
  ggplot(data = dataset, mapping = aes(x = {{ xvar }}, y = {{ yvar }})) +
    geom_point(size = pt_size) +
    geom_smooth()
}

make_plot(dataset = mpg, xvar = cty, yvar = hwy)
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



I often wish it were easier to get my own custom summary statistics for numeric variables in EDA rather than using `mosaic::favstats()`. Using `group_by()` and `summarise()` from the tidyverse reads clearly but takes so many lines, but if I only had to write the code once...

```
summary6 <- function(data, var) {
  data |> summarize(
    mean = mean({{ var }}, na.rm = TRUE),
    median = median({{ var }}, na.rm = TRUE),
    sd = sd({{ var }}, na.rm = TRUE),
    IQR = IQR({{ var }}, na.rm = TRUE),
    n = n(),
    n_miss = sum(is.na({{ var }})),
    .groups = "drop"    # to leave the data in an ungrouped state
  )
}
```

```
mpg |> summary6(hwy)
```

```
# A tibble: 1 x 6
  mean median    sd  IQR     n n_miss
<dbl> <dbl> <dbl> <dbl> <int> <int>
1  23.4     24  5.95     9   234     0
```

Even cooler, I can use my new function with `group_by()`!

```
mpg |>
  group_by(drv) |>
  summary6(hwy)
```

```
# A tibble: 3 x 7
  drv      mean median    sd   IQR     n n_miss
<chr> <dbl>  <dbl> <dbl> <dbl> <int> <int>
1 4      19.2    18  4.08     5   103     0
2 f      28.2    28  4.21     3   106     0
3 r      21     21  3.66     7    25     0
```

You can even pass conditions into a function using the embrace:

[Pause to Ponder:] Predict what the code below will do, and (only) then run it to check. Think about: why do we have `sort = sort`? why not embrace `df`? why didn't we need `n` in the arguments?

`sort = sort`, because we already defined it in our argument that the default is `TRUE`. `count()` already has a built in `sort` argument.

We are not embracing `df` because `embrace` is only for column name. Our dataframe which is in the form of tibble or dataframe is already a regular object that R recognise.

we do not need `n` because `count` will automatically make it for us.

```
new_function <- function(df, var, condition, sort = TRUE) {
  df |>
    filter({{ condition }}) |>
    count({{ var }}, sort = sort) |>
    mutate(prop = n / sum(n))
}

mpg |> new_function(var = manufacturer,
                   condition = manufacturer %in% c("audi",
                                                    "honda",
                                                    "hyundai",
                                                    "nissan",
                                                    "subaru",
                                                    "toyota",
                                                    "volkswagen")
                   )
```

Data-masking vs. tidy-selection (Section 25.3.4)

Why doesn't the following code work?

```
count_missing <- function(df, group_vars, x_var) {  
  df |>  
    group_by({{ group_vars }}) |>  
    summarize(  
      n_miss = sum(is.na({{ x_var }})),  
      .groups = "drop"  
    )  
}  
  
flights |>  
  count_missing(c(year, month, day), dep_time)
```

```
Error in `group_by()`:  
i In argument: `c(year, month, day)`.  
Caused by error:  
! `c(year, month, day)` must be size 336776 or 1, not 1010328.
```

The problem is that `group_by()` uses data-masking rather than tidy-selection; it is selecting certain variables rather than evaluating values of those variables. These are the two most common subtypes of tidy evaluation:

- Data-masking is used in functions like `arrange()`, `filter()`, `mutate()`, and `summarize()` that compute with variables. Data masking is an R feature that blends programming variables that live inside environments (env-variables) with statistical variables stored in data frames (data-variables).
- Tidy-selection is used for functions like `select()`, `relocate()`, and `rename()` that select variables. Tidy selection provides a concise dialect of R for selecting variables based on their names or properties.

More detail can be found [here](#).

The error above can be solved by using the `pick()` function, which uses tidy selection inside of data masking:

```
count_missing <- function(df, group_vars, x_var) {  
  df |>  
    group_by(pick({{ group_vars }})) |>
```

```

      summarize(
        n_miss = sum(is.na({{ x_var }})),
        .groups = "drop"
      )
}

flights |>
  count_missing(c(year, month, day), dep_time)

```

```

# A tibble: 365 x 4
   year month   day n_miss
  <int> <int> <int>   <int>
1  2013     1     1       4
2  2013     1     2       8
3  2013     1     3      10
4  2013     1     4       6
5  2013     1     5       3
6  2013     1     6       1
7  2013     1     7       3
8  2013     1     8       4
9  2013     1     9       5
10 2013     1    10       3
# i 355 more rows

```

[Pause to Ponder:] Here’s another nice use of `pick()`. Predict what the function will do, then run the code to see if you are correct.

`pick()` provides a way to easily select a subset of columns from our data while inside a “data masking” function such as `mutate()` or `summarise()`

```

# Source: https://twitter.com/pollicipes/status/1571606508944719876
new_function <- function(data, rows, cols) {
  data |>
    count(pick(c({{ rows }}, {{ cols }}))) |>
    pivot_wider(
      names_from = {{ cols }},
      values_from = n,
      names_sort = TRUE,
      values_fill = 0
    )
}

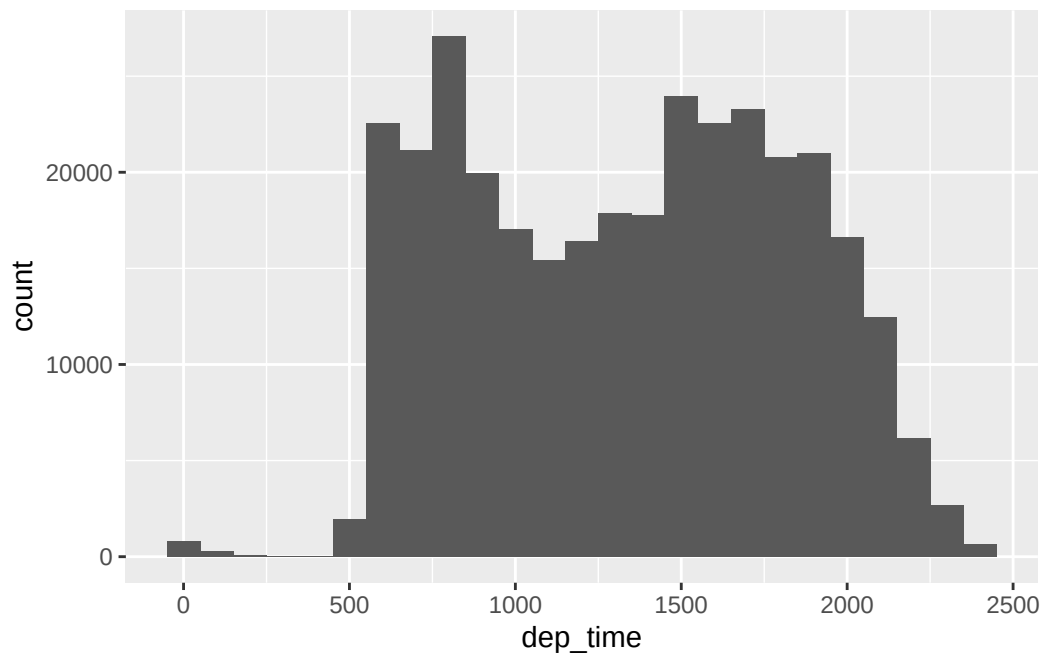
```

```
mpg |> new_function(c(manufacturer, model), cyl)
```

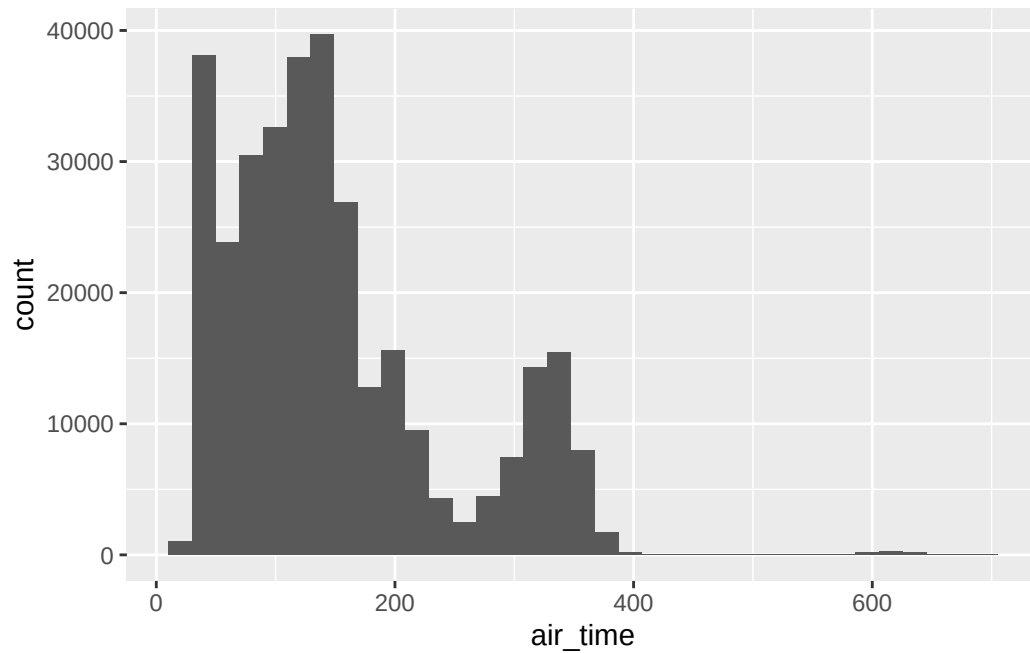
Plot functions

Let's say you find yourself making a lot of histograms:

```
flights |>  
  ggplot(aes(x = dep_time)) +  
  geom_histogram(bins = 25)
```

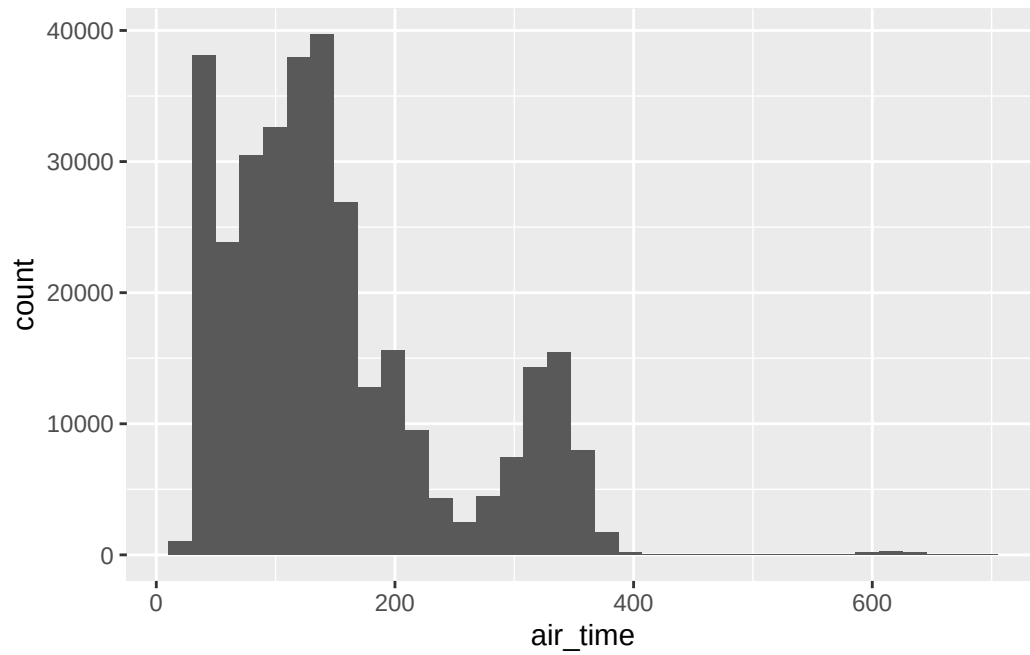


```
flights |>  
  ggplot(aes(x = air_time)) +  
  geom_histogram(bins = 35)
```



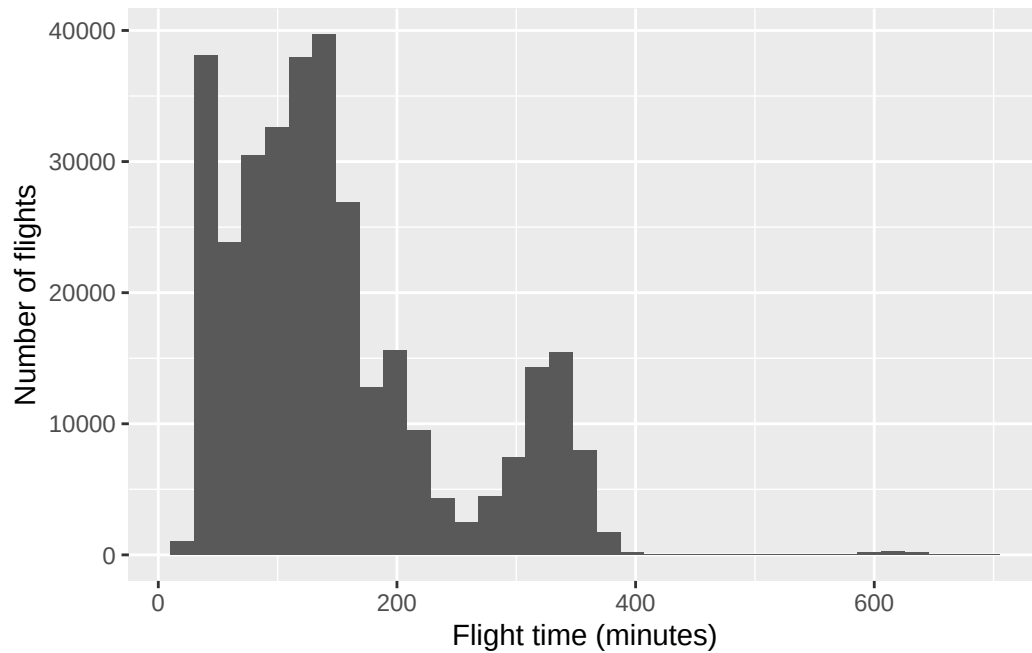
Just use embrace to create a histogram-making function

```
histogram <- function(df, var, bins = NULL) {  
  df |>  
    ggplot(aes(x = {{ var }})) +  
    geom_histogram(bins = bins)  
}  
  
flights |> histogram(air_time, 35)
```



Since `histogram()` returns a `ggplot`, you can add any layers you want

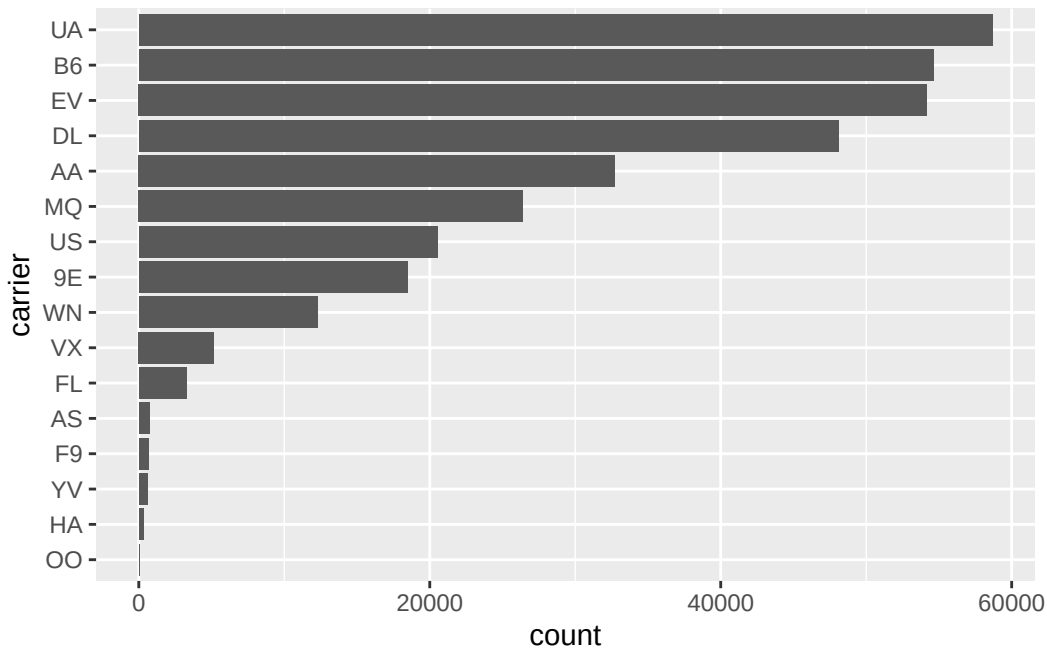
```
flights |>
  histogram(air_time, 35) +
  labs(x = "Flight time (minutes)", y = "Number of flights")
```



You can also combine data wrangling with plotting. Note that we need the “walrus operator” (`:=`) since the variable name on the left is being generated with user-supplied data.

```
# sort counts with highest values at top and counts on x-axis
sorted_bars <- function(df, var) {
  df |>
    mutate({{ var }} := fct_rev(fct_infreq({{ var }}})) |>
    ggplot(aes(y = {{ var }})) +
    geom_bar()
}

flights |> sorted_bars(carrier)
```

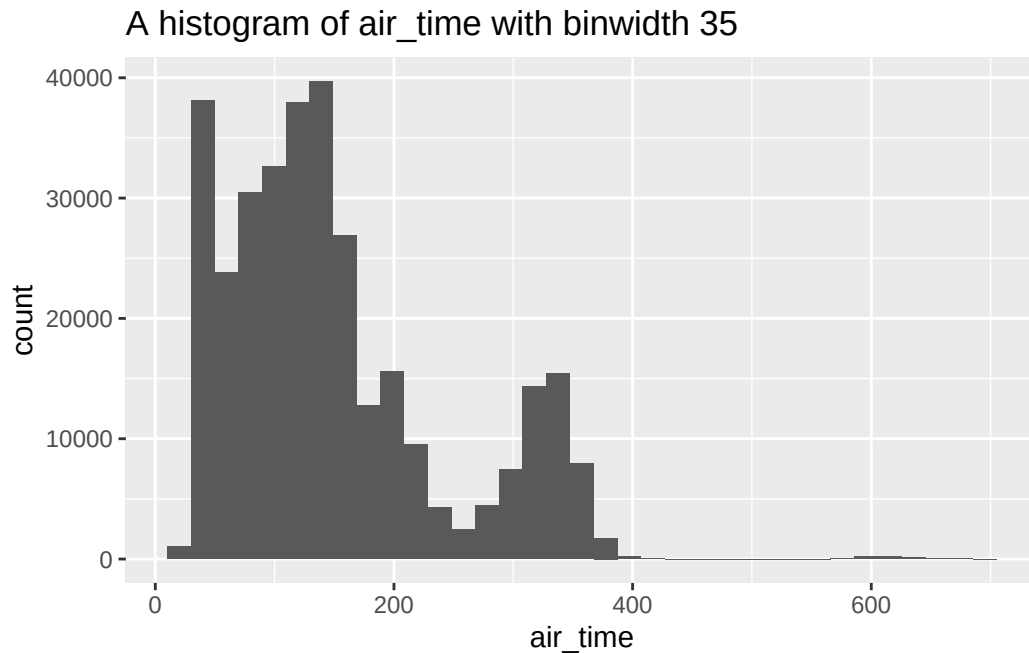


Finally, it would be really helpful to label plots based on user inputs. This is a bit more complicated, but still do-able!

For this, we'll need the `rlang` package. `rlang` is a low-level package that's used by just about every other package in the tidyverse because it implements tidy evaluation (as well as many other useful tools).

Check out the following update of our `histogram()` function which uses the `engluce()` function from the `rlang` package:

```
histogram <- function(df, var, bins) {  
  label <- rlang::engluce("A histogram of {{var}} with binwidth {bins}")  
  
  df |>  
    ggplot(aes(x = {{ var }})) +  
    geom_histogram(bins = bins) +  
    labs(title = label)  
}  
  
flights |> histogram(air_time, 35)
```



On Your Own

```
screenshare <- function(names) {  
  sample(names, 1)  
}  
  
screenshare(c("Mary", "Isimbi", "Aitha", "Lelo", "Cynthia", "Kaisa"))
```

```
[1] "Mary"
```

1. Rewrite this code snippet as a function: `x / sum(x, na.rm = TRUE)`. This code creates weights which sum to 1, where NA values are ignored. Test it for at least two different vectors. (Make sure at least one has NAs!)

```
weights <- function(x) {  
  x / sum(x, na.rm = TRUE)  
}  
  
x <- c(4, 6, 8, 9)  
t <- c(4, 6, 8, 9, NA)
```

```
weights(x)
```

```
[1] 0.1481481 0.2222222 0.2962963 0.3333333
```

```
weights(y)
```

```
[1] 0.08333333 0.25000000 0.12500000 0.33333333 0.20833333
```

2. Create a function to calculate the standard error of a variable, where $SE = \text{square root of the variance divided by the sample size}$. Hint: start with a vector like `x <- 0:5` or `x <- gss_cat$age` and write code to find the SE of `x`, then turn it into a function to handle any vector `x`. Note: `var` is the function to find variance in R and `sqrt` does square root. `length` may also be handy. Test your function on two vectors that do not include NAs (i.e. do **not** worry about removing NAs at this point).

```
se_fun <- function(x) {  
  sqrt(var(x))/length(x)  
}
```

```
x <- c(4, 6, 8, 9)
```

```
se_fun(x)
```

```
[1] 0.5543389
```

3. Use your `se` function within `summarize` to get a table of the mean and s.e. of `hwy` and `cty` by `class` in the `mpg` dataset.

```
se_fun1 <- function(x) {  
  sqrt(var(x))/length(x)  
}
```

```
mpg |>  
  group_by(class) |>  
  summarise(meanhwy = mean(hwy),  
            meancty = mean(cty),  
            se_fun_hwy = se_fun1(hwy),  
            se_fun_cty = se_fun1(cty))
```

```
# A tibble: 7 x 5
  class      meanhwy meancty se_fun_hwy se_fun_cty
  <chr>      <dbl>   <dbl>   <dbl>   <dbl>
1 2seater    24.8    15.4    0.261    0.110
2 compact    28.3    20.1    0.0805   0.0720
3 midsize    27.3    18.8    0.0521   0.0475
4 minivan    22.4    15.8    0.188    0.167
5 pickup     16.9    13      0.0689   0.0620
6 subcompact 28.1    20.4    0.154    0.131
7 suv        18.1    13.5    0.0480   0.0390
```

4. Use your `se` function within `summarize` to get a table of the mean and s.e. of `arr_delay` and `dep_delay` by carrier in the `flights` dataset. Why does the output look like this?

```
se_na <- function(x, includeNAs) {
  if (includeNAs) {
    x <- na.omit(x)
  }
  sqrt(var(x) / length(x))
}

se_na(c(1:5, NA), includeNAs = TRUE)
```

```
[1] 0.7071068
```

```
flights |>
  group_by(carrier) |>
  summarize(mean_arr_delay = mean(arr_delay, na.rm = TRUE),
            mean_dep_delay = mean(dep_delay, na.rm = TRUE),
            se_arr = se_na(arr_delay, includeNAs = TRUE),
            se_dep = se_na(dep_delay, includeNAs = TRUE))
```

```
# A tibble: 16 x 5
  carrier mean_arr_delay mean_dep_delay se_arr se_dep
  <chr>      <dbl>         <dbl>   <dbl> <dbl>
1 9E         7.38         16.7    0.381 0.348
2 AA         0.364         8.59    0.238 0.209
3 AS        -9.93         5.80    1.37  1.18
4 B6         9.46        13.0    0.184 0.165
5 DL         1.64         9.26    0.203 0.182
6 EV        15.8        20.0    0.221 0.205
```

7 F9	21.9	20.2	2.36	2.23
8 FL	20.1	18.7	0.960	0.933
9 HA	-6.92	4.90	4.06	4.01
10 MQ	10.8	10.6	0.273	0.247
11 OO	11.9	12.6	9.02	8.00
12 UA	3.56	12.1	0.170	0.148
13 US	2.13	3.78	0.235	0.199
14 VX	1.76	12.9	0.699	0.626
15 WN	9.65	17.7	0.427	0.394
16 YV	15.6	19.0	2.27	2.11

5. Make your `se` function handle NAs with an `na.rm` option. Test your new function (you can call it `se` again) on a vector that doesn't include NA and on the same vector with an added NA. **Be sure to check that it gives the expected output with `na.rm = TRUE` and `na.rm = FALSE`.** Make `na.rm = FALSE` the default value. Repeat #4. (Hint: be sure when you divide by sample size you don't count any NAs)

```
se_na_5 <- function(x, na.rm = FALSE) {          # MAKING na.rm = FALSE the default value.
  if (na.rm) {
    x <- na.omit(x)
  }
  sqrt(var(x) / length(x))
}

hio <- x <- c(10, 12, 13, 15)
hi <- c(10, 12, 13, 15, NA)

se_na_5(hio)
```

```
[1] 1.040833
```

```
se_na_5(hi)
```

```
[1] NA
```

```
se_na_5(hi, na.rm = TRUE)
```

```
[1] 1.040833
```

6. Create `both_na()`, a function that takes two vectors of the same length and returns how many positions have an NA in both vectors. Hint: create two vectors like `test_x <- c(1, 2, 3, NA, NA)` and `test_y <- c(NA, 1, 2, 3, NA)` and write code that works for `test_x` and `test_y`, then turn it into a function that can handle any `x` and `y`. (In this case, the answer would be 1, since both vectors have NA in the 5th position.) Test it for at least one more combination of `x` and `y`.

```
both_na <- function (x, y) {  
  sum(is.na(x) & is.na(y))  
}
```

```
x <- c(1, 2, 3, NA, NA)  
y <- c(NA, 1, 2, 3, NA)
```

```
both_na(x,y)
```

```
[1] 1
```

```
x1 <- c(1, 1, 3, NA, NA, NA, NA)  
y1 <- c(NA, 1, 2, 3, NA, NA, NA)
```

```
both_na(x1, y1)
```

```
[1] 3
```

```
test_x <- c(1, 2, 3, NA, NA, NA)  
test_y <- c(NA, 1, 2, 3, NA)  
both_na(test_x, test_y)
```

Warning in `is.na(x) & is.na(y)`: longer object length is not a multiple of shorter object length

```
[1] 2
```

7. Run your code from (6) with the following two vectors: `test_x <- c(1, 2, 3, NA, NA, NA)` and `test_y <- c(NA, 1, 2, 3, NA)`. Did you get the output you wanted or expected? Modify your function using `if`, `else`, and `stop` to print an error if `x` and `y` are not the same length. Then test again with `test_x`, `test_y` and the sets of vectors you used in (6).

We get a warning message “longer object length is not a multiple of shorter object length”.

```
both_na_7 <- function (x, y) {  
  if (length(x) != length(y)) {  
    stop("'x' and 'y' must be the same length", call. = FALSE)  
  } else {sum(is.na(x) & is.na(y))} # WE WANT & SIGN AND NOT == S  
}  
  
x <- c(1, 2, 3, NA, NA)  
y <- c(NA, 1, 2, 3, NA)  
  
both_na_7(x,y)
```

```
[1] 1
```

```
x1 <- c(1, 1, 3, NA, NA, NA, NA)  
y1 <- c(NA, 1, 2, 3, NA, NA, NA)  
  
both_na_7(x1, y1)
```

```
[1] 3
```

```
test_x <- c(1, 2, 3, NA, NA, NA)  
test_y <- c(NA, 1, 2, 3, NA)  
  
both_na_7(test_x, test_y)
```

Error:

```
! 'x' and 'y' must be the same length
```

8. Here is a way to get not_cancelled flights in the flights dataset:

```
not_cancelled <- flights |>  
  filter(!is.na(dep_delay), !is.na(arr_delay))  
  
flights |>  
  filter(is.na(dep_delay), is.na(arr_delay))
```

```
# A tibble: 8,255 x 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     NA           1630           NA     NA           1815
2  2013     1     1     NA           1935           NA     NA           2240
3  2013     1     1     NA           1500           NA     NA           1825
4  2013     1     1     NA            600           NA     NA            901
5  2013     1     2     NA           1540           NA     NA           1747
6  2013     1     2     NA           1620           NA     NA           1746
7  2013     1     2     NA           1355           NA     NA           1459
8  2013     1     2     NA           1420           NA     NA           1644
9  2013     1     2     NA           1321           NA     NA           1536
10 2013     1     2     NA           1545           NA     NA           1910
# i 8,245 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
not_cancelled
```

```
# A tibble: 327,346 x 19
  year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
1  2013     1     1     517           515            2     830           819
2  2013     1     1     533           529            4     850           830
3  2013     1     1     542           540            2     923           850
4  2013     1     1     544           545           -1    1004          1022
5  2013     1     1     554           600           -6     812           837
6  2013     1     1     554           558           -4     740           728
7  2013     1     1     555           600           -5     913           854
8  2013     1     1     557           600           -3     709           723
9  2013     1     1     557           600           -3     838           846
10 2013     1     1     558           600           -2     753           745
# i 327,336 more rows
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Is it necessary to check is.na for both departure and arrival? Using summarize, find the number of flights missing departure delay, arrival delay, and both. (Use your new function!)

```
flights |>
  summarise(n_dep_delay = sum(is.na(dep_delay)),
            n_arr_delay = sum(is.na(arr_delay)),
            n_both = both_na_7(dep_delay, arr_delay)
  )
```

```
# A tibble: 1 x 3
  n_dep_delay n_arr_delay n_both
    <int>      <int>    <int>
1      8255        9430     8255
```

9. Read the code for each of the following three functions, puzzle out what they do, and then brainstorm better names.

```
f1 <- function(time1, time2) {
  hour1 <- time1 %/% 100
  min1 <- time1 %% 100
  hour2 <- time2 %/% 100
  min2 <- time2 %% 100

  (hour2 - hour1)*60 + (min2 - min1)
}

f2 <- function(lengthcm, widthcm) {
  (lengthcm / 2.54) * (widthcm / 2.54)
}

f3 <- function(x) {
  fct_collapse(x, "non answer" = c("No answer", "Refused",
                                   "Don't know", "Not applicable"))
}
```

f1 is to find the duration or length in minutes between two given times. f1 can be named : duration_mins

f2 is to turn cm to inch. f2 can be named : to_inches

f3 is to recode several categories into one general group “non answer” f3 can be named to : group_to_na

10. Explain what the following function does and demonstrate by running `foo1(x)` with a few appropriately chosen vectors `x`. (Hint: set `x` and run the “guts” of the function piece by piece.)

```
foo1 <- function(x) {
  diff <- x[-1] - x[1:(length(x) - 1)]
  sum(diff < 0)
}

foo <- function(x) {
  x[-1] - x[1:(length(x) - 1)]
}

foo(c(8, 15, 13, 12, -100, -101))
```

```
[1]      7    -2    -1 -112    -1
```

```
# my answer:
# so, 15-8 = 7; 13-15 = -2
# this first part of the function takes the difference of a value from x from the value that
# so like (n+1 - n) kind of thing.

# so automatically the second part of the code
# which is sum(diff < 0)
# counts how many times negative difference appear...
```

11. The `foo1()` function doesn't perform well if a vector has missing values. Amend `foo1()` so that it produces a helpful error message and stops if there are any missing values in the input vector. Show that it works with appropriately chosen vectors `x`. Be sure you add `error = TRUE` to your R chunk, or else knitting will fail!

```
foo2 <- function(x) {
  if (is.na(sum(x))) {
    stop("there is missing value(s) in the input vector")
  } else {diff <- x[-1] - x[1:(length(x) - 1)]
  sum(diff < 0)
  }
}

foo2(c(8, 15, 13, 12, NA, -100, -101))
```

```
Error in `foo2()`:
! there is missing value(s) in the input vector
```

12. Write a function called `greet` using `if`, `else if`, and `else` to print out “good morning” if it’s before 12 PM, “good afternoon” if it’s between 12 PM and 5 PM, and “good evening” if it’s after 5 PM. Your function should work if you input a time like: `greet(time = "2018-05-03 17:38:01 CDT")` or if you input the current time with `greet(time = Sys.time())`. [Hint: check out the `hour` function in the `lubridate` package]

```
greet <- function(time) {  
  h <- hour(time)  
  if (h < 12) {  
    print("good morning :)")  
  } else if (h < 17) {  
    print("good afternoon :)")  
  } else {  
    print("good night :)")  
  }  
}  
  
greet(time = "2018-05-03 16:38:01 CDT")
```

```
[1] "good afternoon :)"
```

```
greet(time = "2018-05-03 04:38:01 CDT")
```

```
[1] "good morning :)"
```

```
greet(time = Sys.time())
```

```
[1] "good night :)"
```

13. Modify the `summary6()` function from earlier to add an argument that gives the user an option to remove missing values, if any exist. Show that your function works for (a) the `hwy` variable in `mpg_tbl <- as_tibble(mpg)`, and (b) the `age` variable in `gss_cat`.

```
summary6_modif <- function(data, var, removeNAs = FALSE) {  
  if (removeNAs) {  
    data <-  
      data |>  
      na.omit({{ var }})  
  }  
  data |>
```

```

    summarize(
      mean = mean({{ var }}),
      median = median({{ var }}),
      sd = sd({{ var }}),
      IQR = IQR({{ var }}),
      n = n(),
      n_miss = sum(is.na({{ var }})),
      .groups = "drop"
    )
  }

```

```

mpg |>
  summary6_modif(hwy, removeNAs = TRUE)

```

```

# A tibble: 1 x 6
   mean median    sd  IQR      n n_miss
<dbl> <dbl> <dbl> <dbl> <int> <int>
1  23.4     24  5.95     9   234     0

```

```

gss_cat |>
  summary6_modif(age, removeNAs = TRUE)

```

```

# A tibble: 1 x 6
   mean median    sd  IQR      n n_miss
<dbl> <int> <dbl> <dbl> <int> <int>
1  47.2     46  17.4    27 11299     0

```

14. Add an argument to (13) to produce summary statistics by group for a second variable (you should now have 4 possible inputs to your function). Show that your function works for (a) the `hwy` variable in `mpg_tbl <- as_tibble(mpg)` grouped by `drv`, and (b) the `age` variable in `gss_cat` grouped by `partyid`.

```

summary6_modif2 <- function(data, group_vars, var, removeNAs = FALSE) {
  if (removeNAs) {
    data <-
      data |>
        na.omit({{ var }})
  }
  data |>
    group_by({{ group_vars }}) |>

```

```

    summarize(
      mean = mean({{ var }}),
      median = median({{ var }}),
      sd = sd({{ var }}),
      IQR = IQR({{ var }}),
      n = n(),
      n_miss = sum(is.na({{ var }})),
      .groups = "drop"
    )
  }

mpg |>
  summary6_modif2(drv, hwy, removeNAs = TRUE)

```

```

# A tibble: 3 x 7
  drv    mean median    sd  IQR    n n_miss
<chr> <dbl>  <dbl> <dbl> <dbl> <int> <int>
1 4      19.2    18  4.08    5   103     0
2 f      28.2    28  4.21    3   106     0
3 r      21     21  3.66    7    25     0

```

```

partyid_table <- gss_cat |>
  summary6_modif2(partyid, age, removeNAs = TRUE)

```

15. Create a function that has a vector as the input and returns the last value. (Note: Be sure to use a name that does not write over an existing function!)

```

last_value <- function(x) {
  x[length(x)]
}

last_value(c(1, 1, 3, NA, NA, NA, NA))

```

[1] NA

16. Save your final table from (14) and write a function to draw a scatterplot of a measure of center (mean or median - user can choose) vs. a measure of spread (sd or IQR - user can choose), with points sized by sample size, to see if there is constant variance. Each point should be labeled with partyid, and the plot title should reflect the variables chosen by the user.

Hint: start with a ggplot with no user input, and then functionize:

```
plotting <- function(df, xvar, yvar, labelvar, sizevar) {  
  ggplot(data = df, mapping = aes(x = {{ xvar }}, y = {{ yvar }})) +  
    geom_point(aes(size = {{ sizevar }})) +  
    geom_smooth(method = lm) +  
    geom_label_repel(aes(label = {{ labelvar }})) +  
    labs(title = "Mean vs IQR")  
}  
  
plotting(partyid_table, mean, IQR, partyid, n)
```